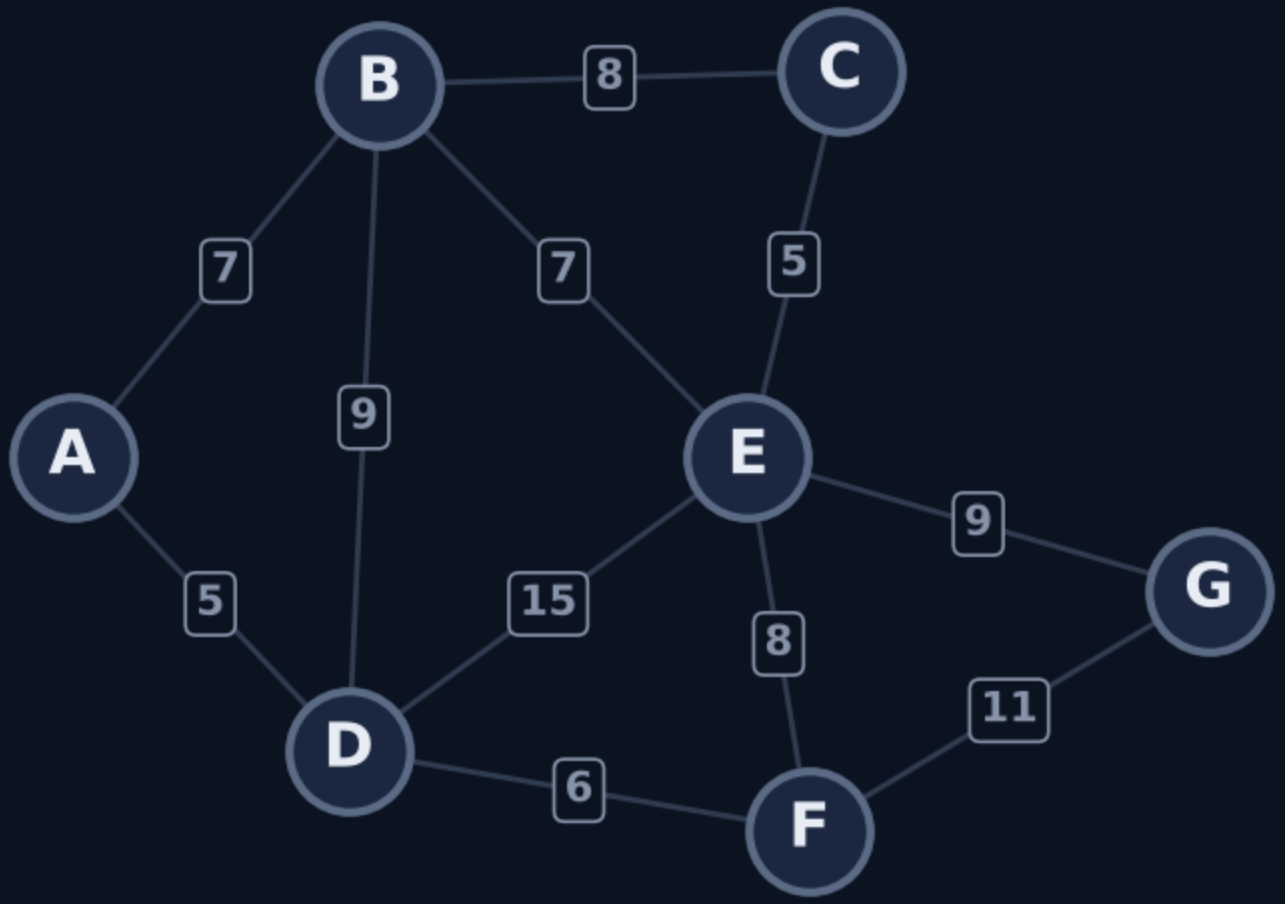
Prim's Algorithm

Building a Minimum Spanning Tree, step by step

A greedy algorithm for weighted, undirected graphs

Hi everyone! Today we'll see how Prim's algorithm finds a minimum spanning tree, one step at a time.

Prim's Algorithm



The goal

Connect every vertex for the smallest total weight, with no cycles.

The result is a Minimum Spanning Tree (MST).

You want every point connected, for the lowest total cost, with no wasteful loops. That cheapest network is called a minimum spanning tree.

How Prim's algorithm works

1. **Start with any vertex.**
   It is now the tree.

2. **Find the frontier.**
   Every edge with one end inside the tree and one end outside.

3. **Add the cheapest frontier edge,**
   together with the new vertex it reaches.

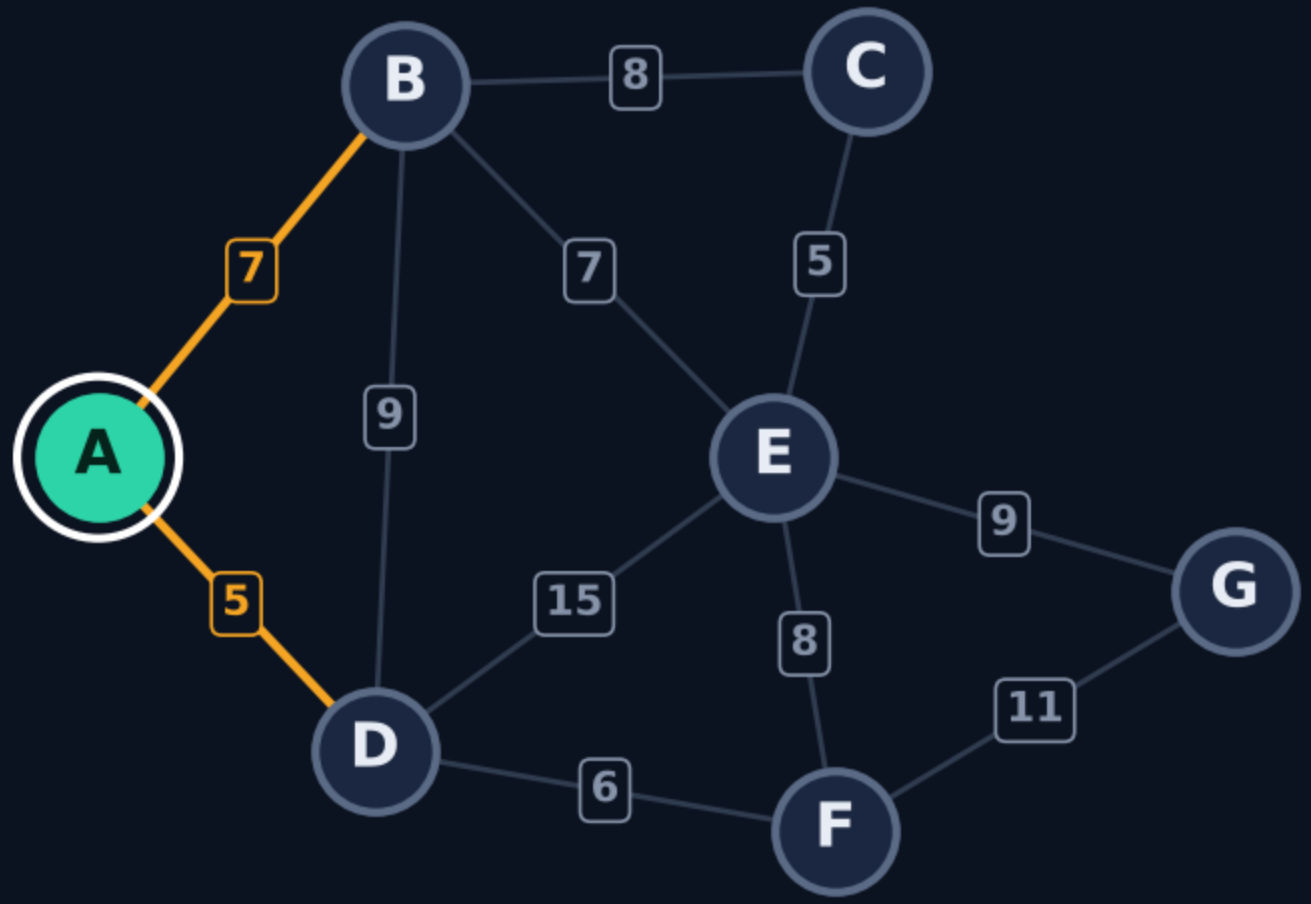
4. **Repeat until every vertex is in the tree.**
   That takes exactly V − 1 edges.

Four. Repeat until every vertex is in the tree.

Prim's Algorithm

Tree 1/7

Start



Start at A

Tree vertices:

A

Tree weight so far:

0

We could start anywhere;

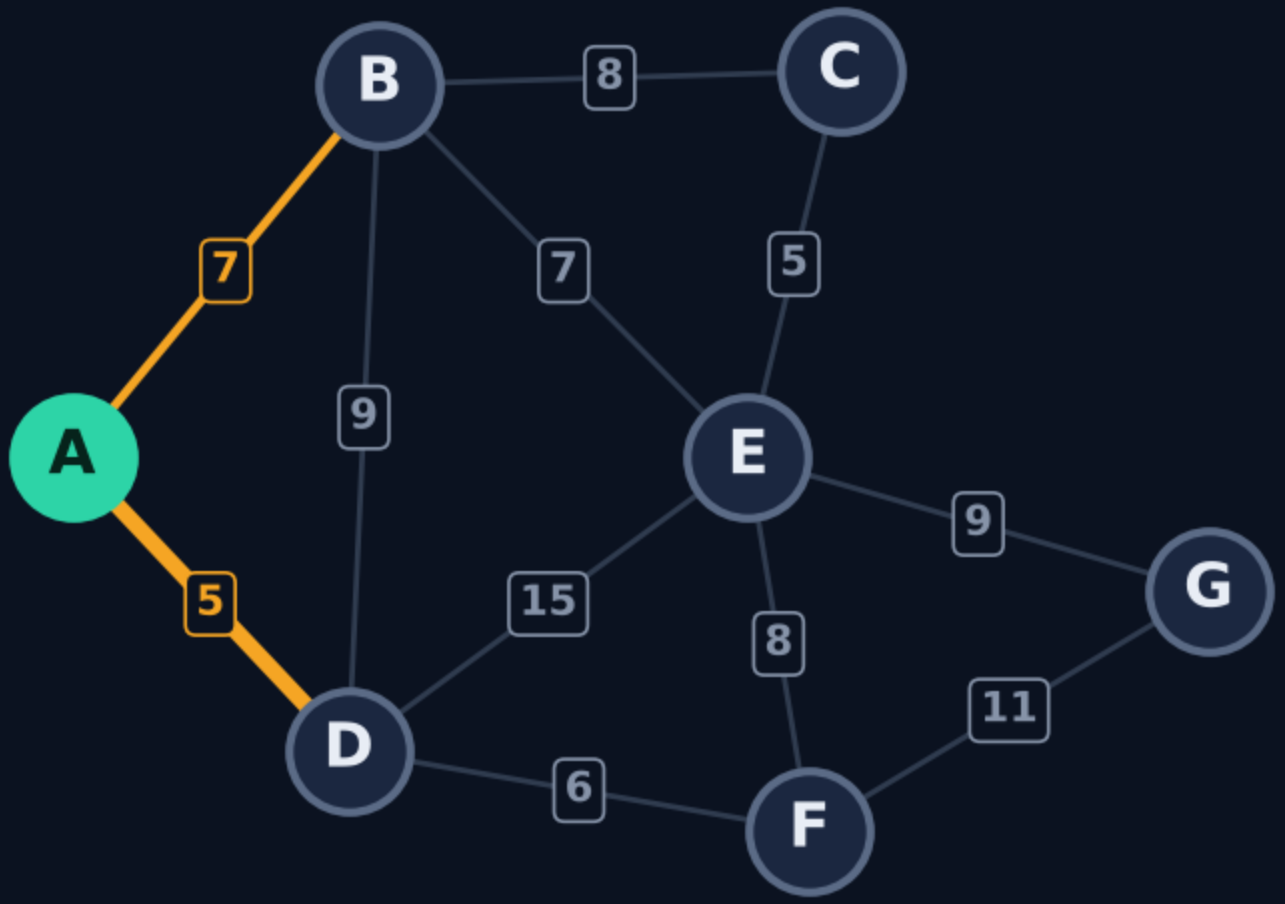
the final total is the same.

Let's try it. We start at vertex A. The tree is just A, and its total weight is zero.

Prim's Algorithm

Tree 1/7

Step 1 of 6



Step 1: look

Tree: A

Frontier edges:

A–D 5 ← cheapest

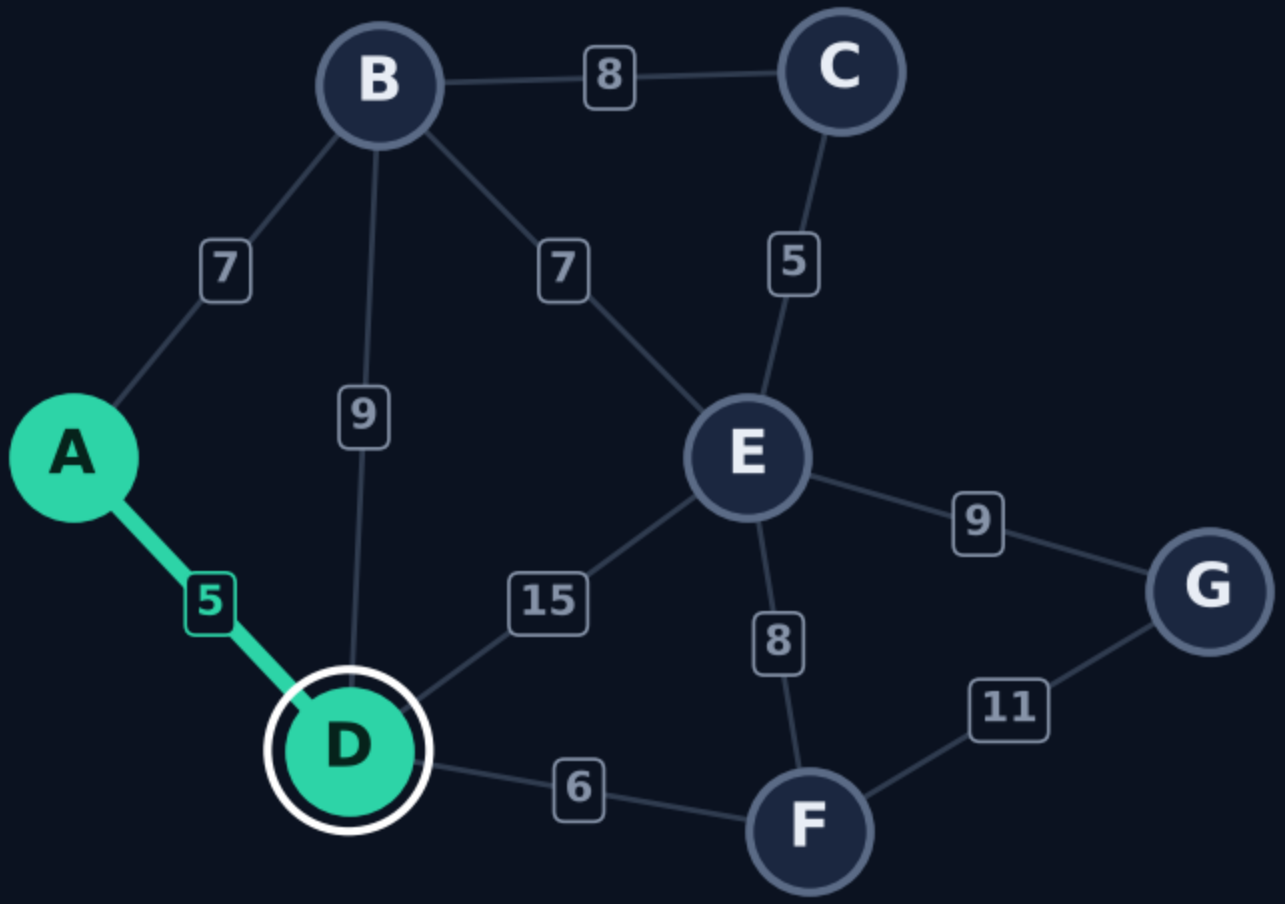
A–B 7

Step one. The frontier has two edges: A to B, seven, and A to D, five. The cheapest is A to D.

Prim's Algorithm

Tree 2/7

Step 1 of 6



Step 1: add

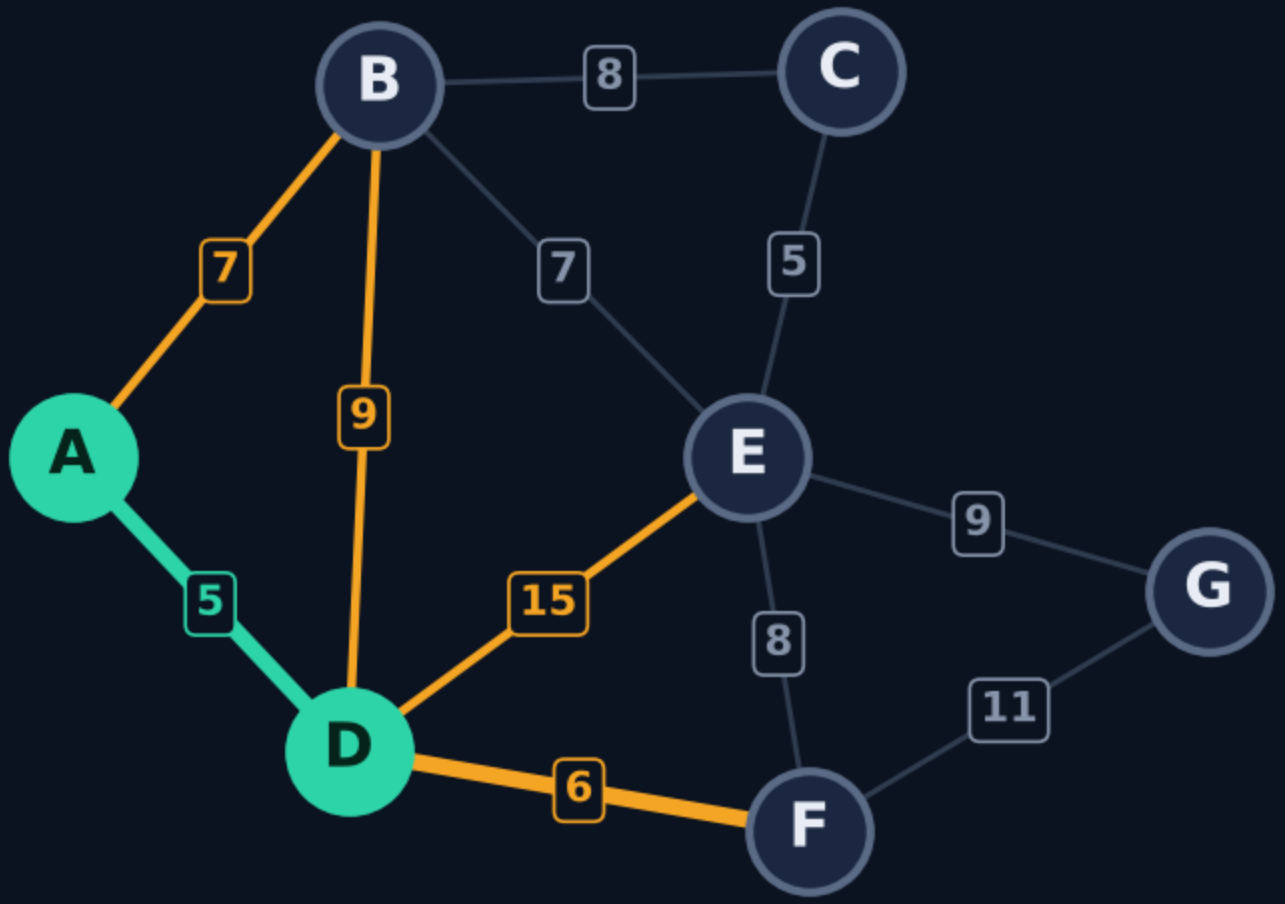
Add edge A–D (weight 5)

Add vertex D to the tree

Total weight so far:

5

We add A to D, and D joins the tree. Total weight so far: five.



Step 2: look

Tree: A, D

Frontier edges:

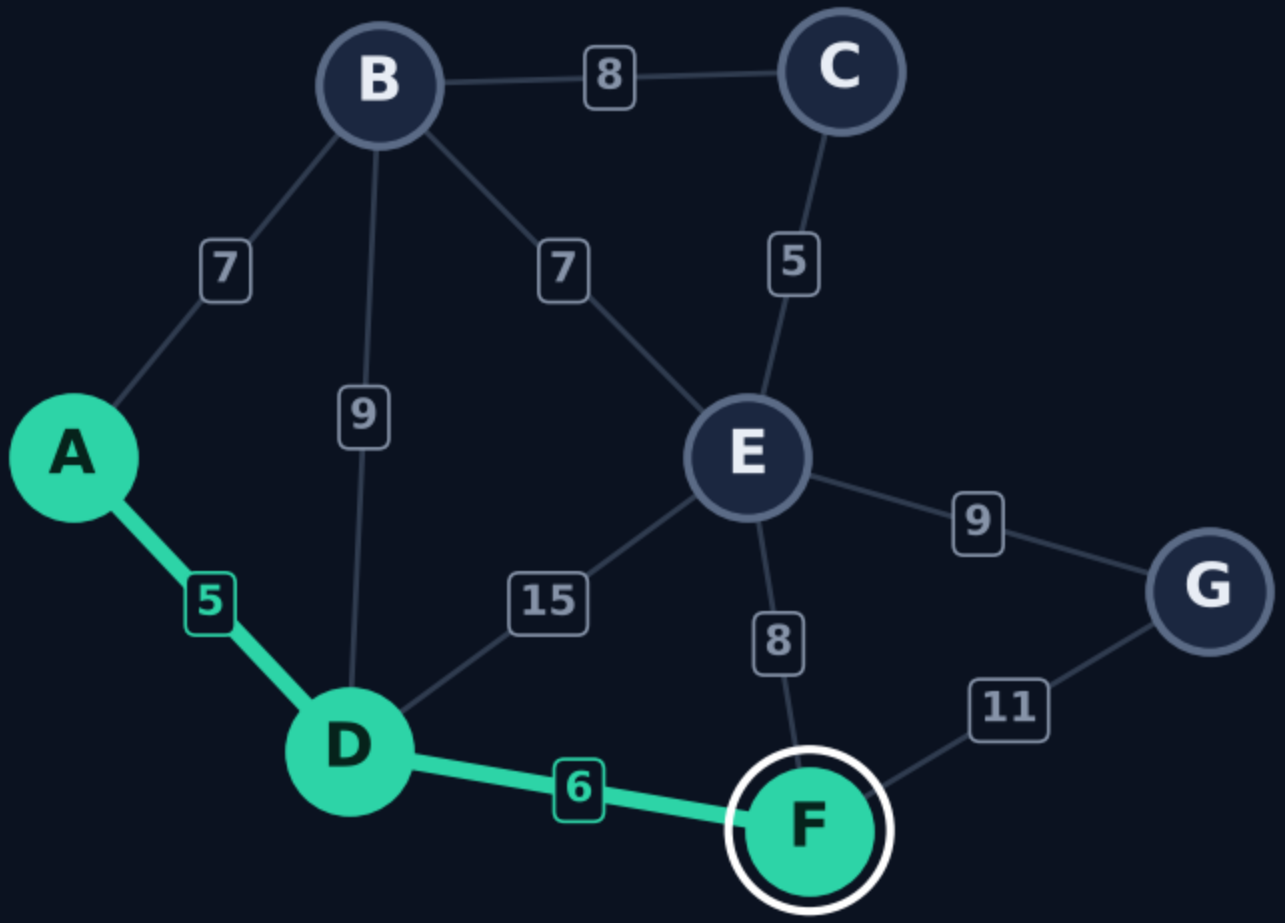
D–F	6	← cheapest
A–B	7	
B–D	9	
D–E	15	

Step two. Now there are four edges on the frontier. The cheapest is D to F, with weight six.

Prim's Algorithm

Tree 3/7

Step 2 of 6



Step 2: add

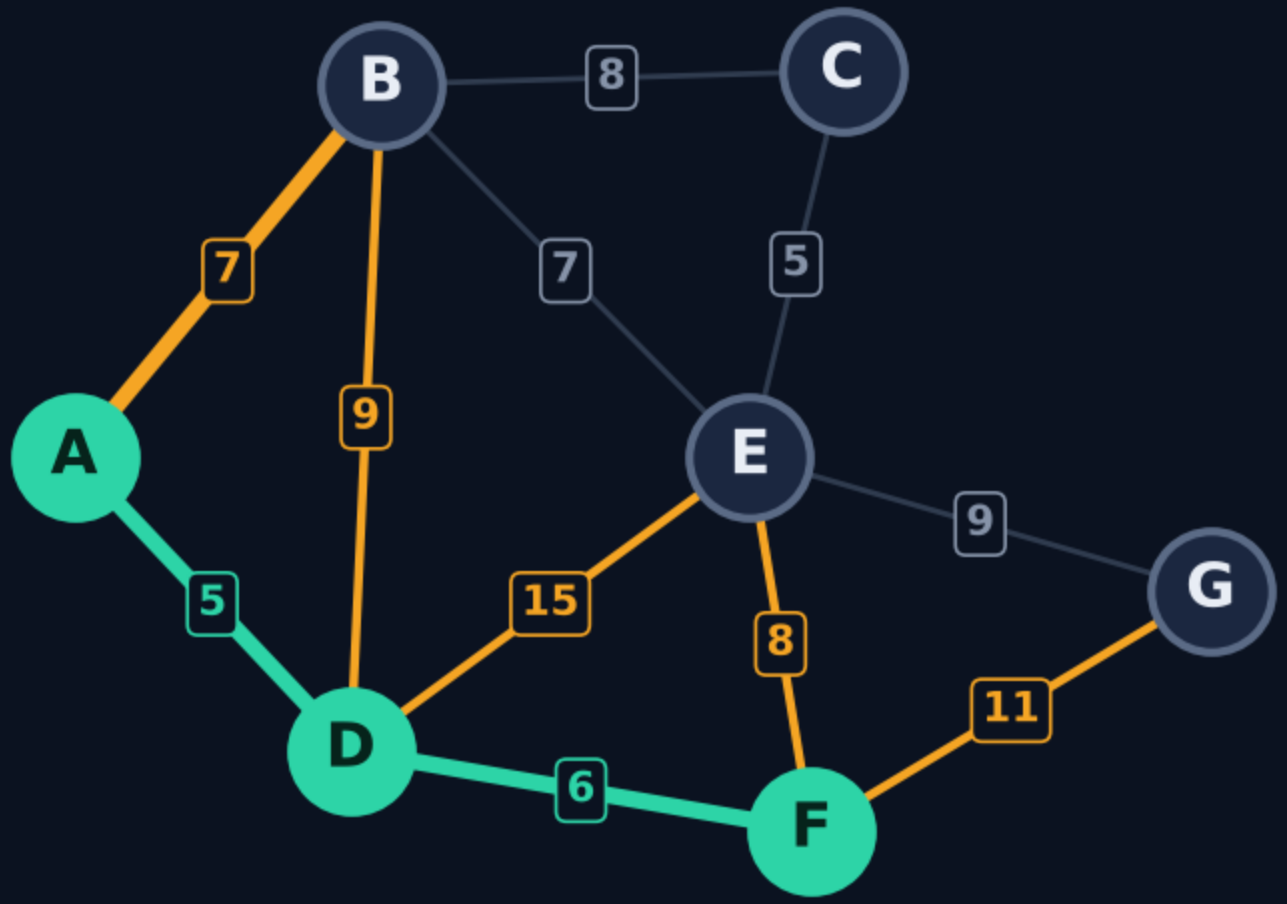
Add edge D–F (weight 6)

Add vertex F to the tree

Total weight so far:

11

We add D to F, and F joins the tree. The total is now eleven.



Step 3: look

Tree: A, D, F

Frontier edges:

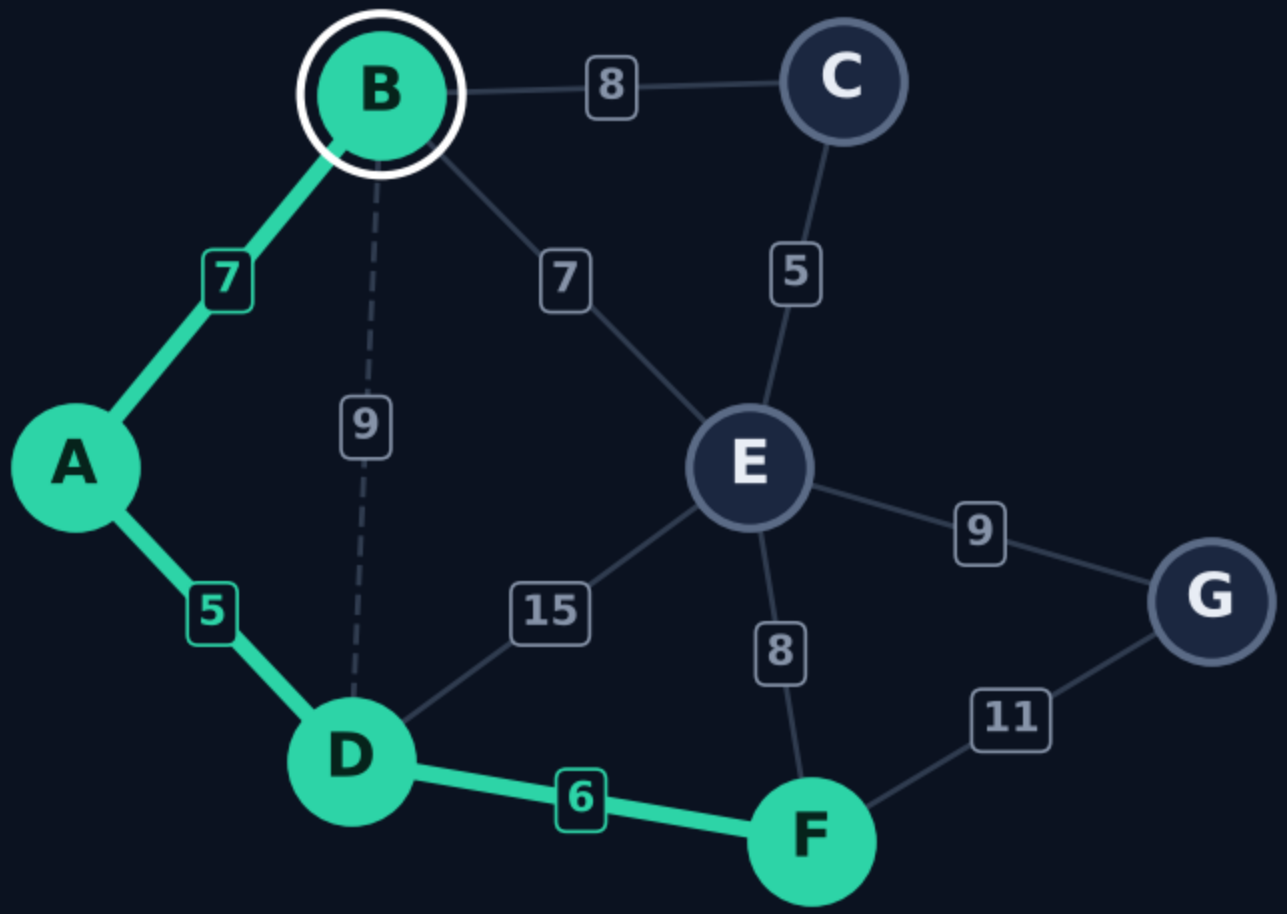
A–B	7	← cheapest
E–F	8	
B–D	9	
F–G	11	
D–E	15	

Step three. Five edges leave the tree, and the cheapest is A to B, with weight seven.

Prim's Algorithm

Tree 4/7

Step 3 of 6



Step 3: add

Add edge A–B (weight 7)

Add vertex B to the tree

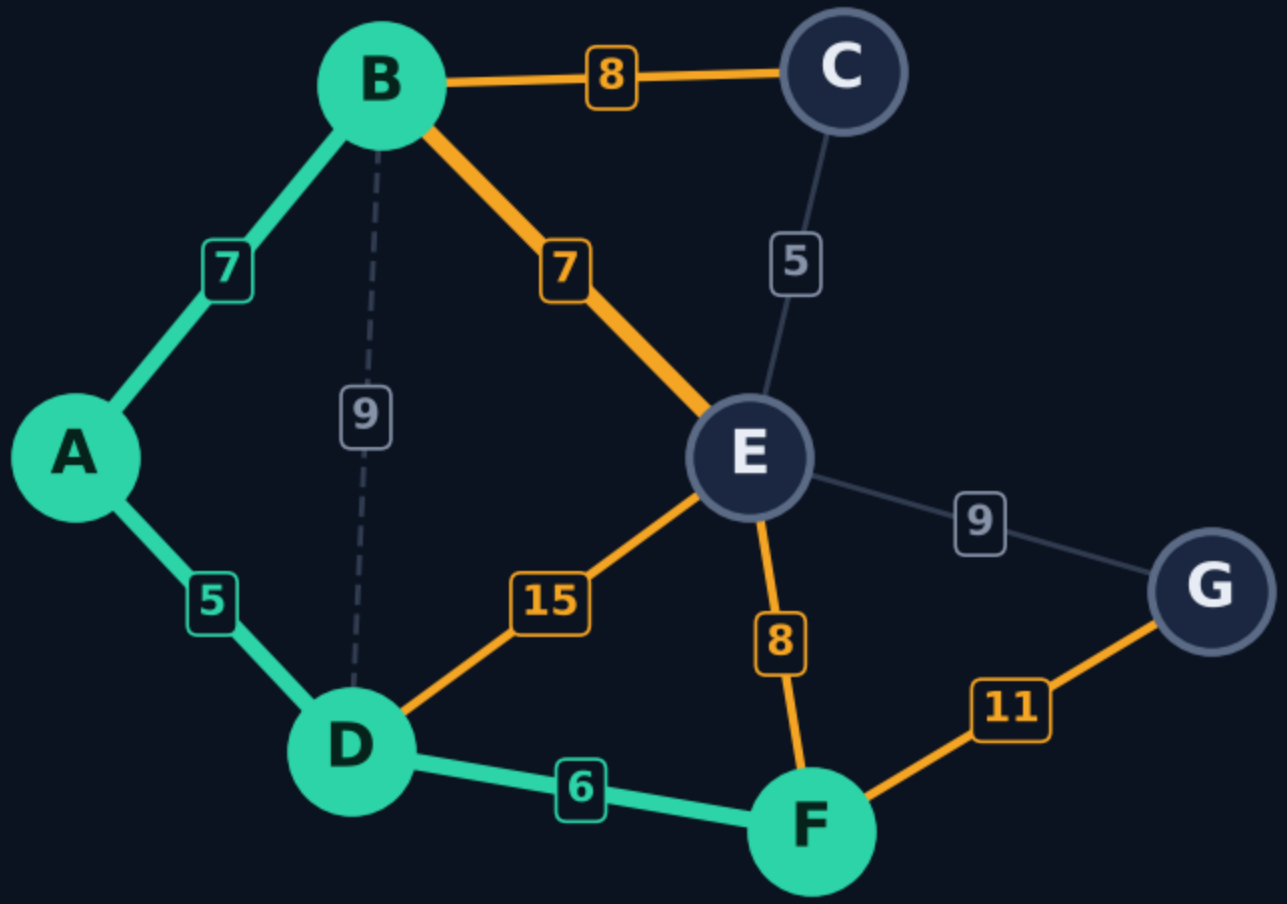
Skipped (would form a cycle):

B–D (9)

Total weight so far:

18

We add A to B. Now B to D connects two vertices that are already in the tree, so it would create a cycle. We skip it. Total: eighteen.



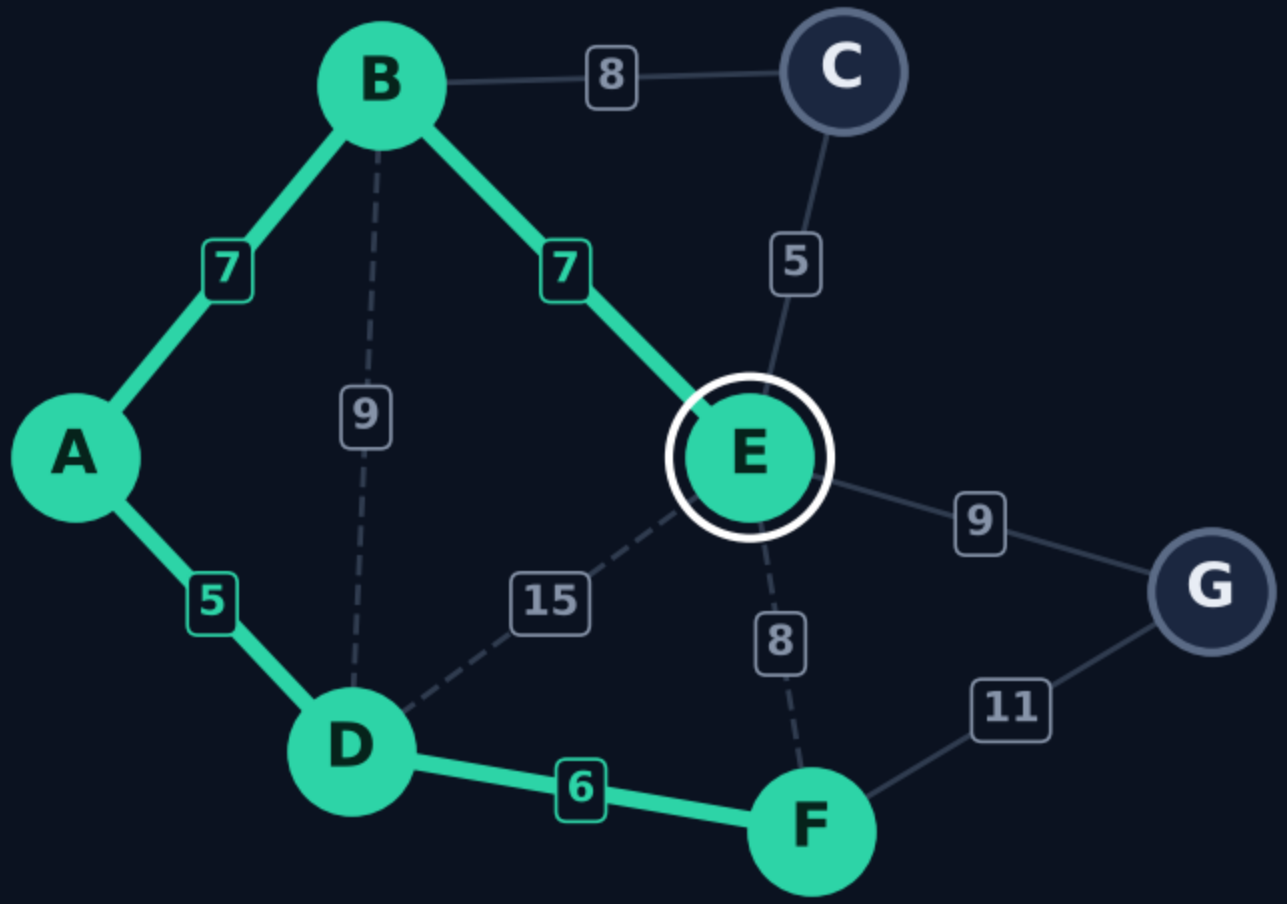
Step 4: look

Tree: A, D, F, B

Frontier edges:

B–E	7	← cheapest
B–C	8	
E–F	8	
F–G	11	
D–E	15	

Step four. Again five frontier edges, and the cheapest is B to E, with weight seven.



Step 4: add

Add edge B–E (weight 7)

Add vertex E to the tree

Skipped (would form a cycle):

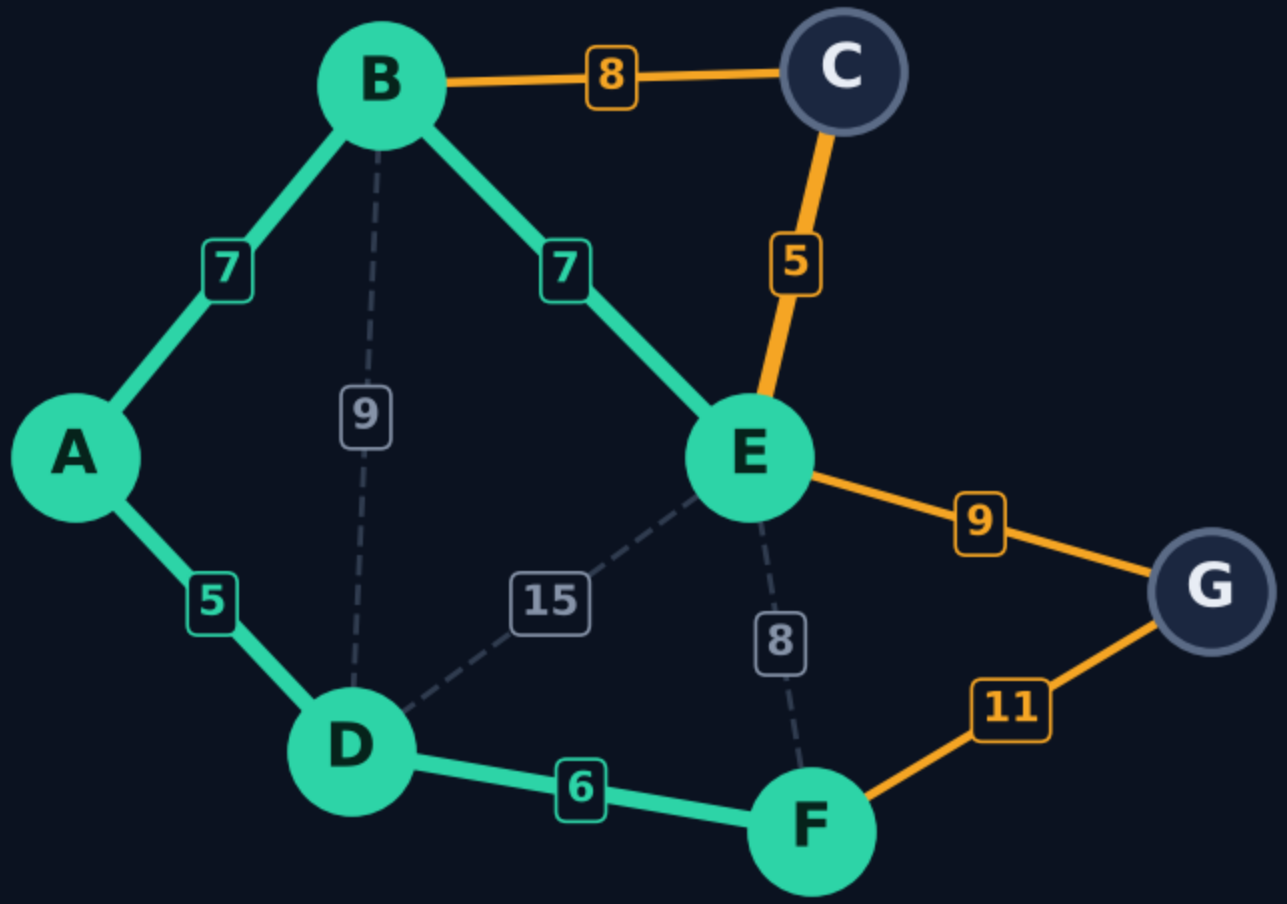
D–E (15)

E–F (8)

Total weight so far:

25

We add B to E. D to E and E to F are now inside the tree as well, so both are skipped. Total: twenty five.



Step 5: look

Tree: A, D, F, B, E

Frontier edges:

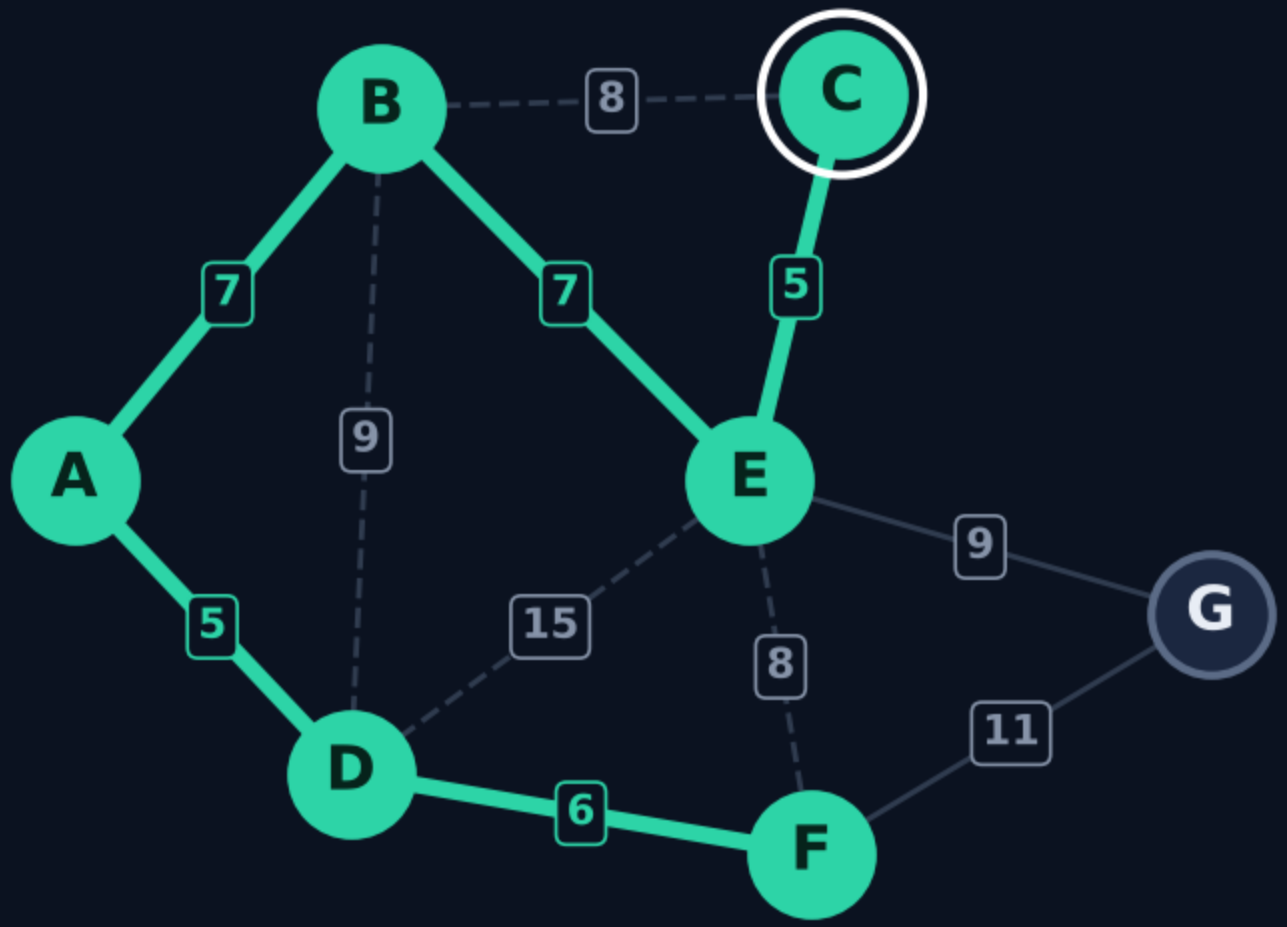
C–E	5	← cheapest
B–C	8	
E–G	9	
F–G	11	

Step five. Four frontier edges. Edge C to E, with weight five, is the cheapest.

Prim's Algorithm

Tree 6/7

Step 5 of 6



Step 5: add

Add edge C–E (weight 5)

Add vertex C to the tree

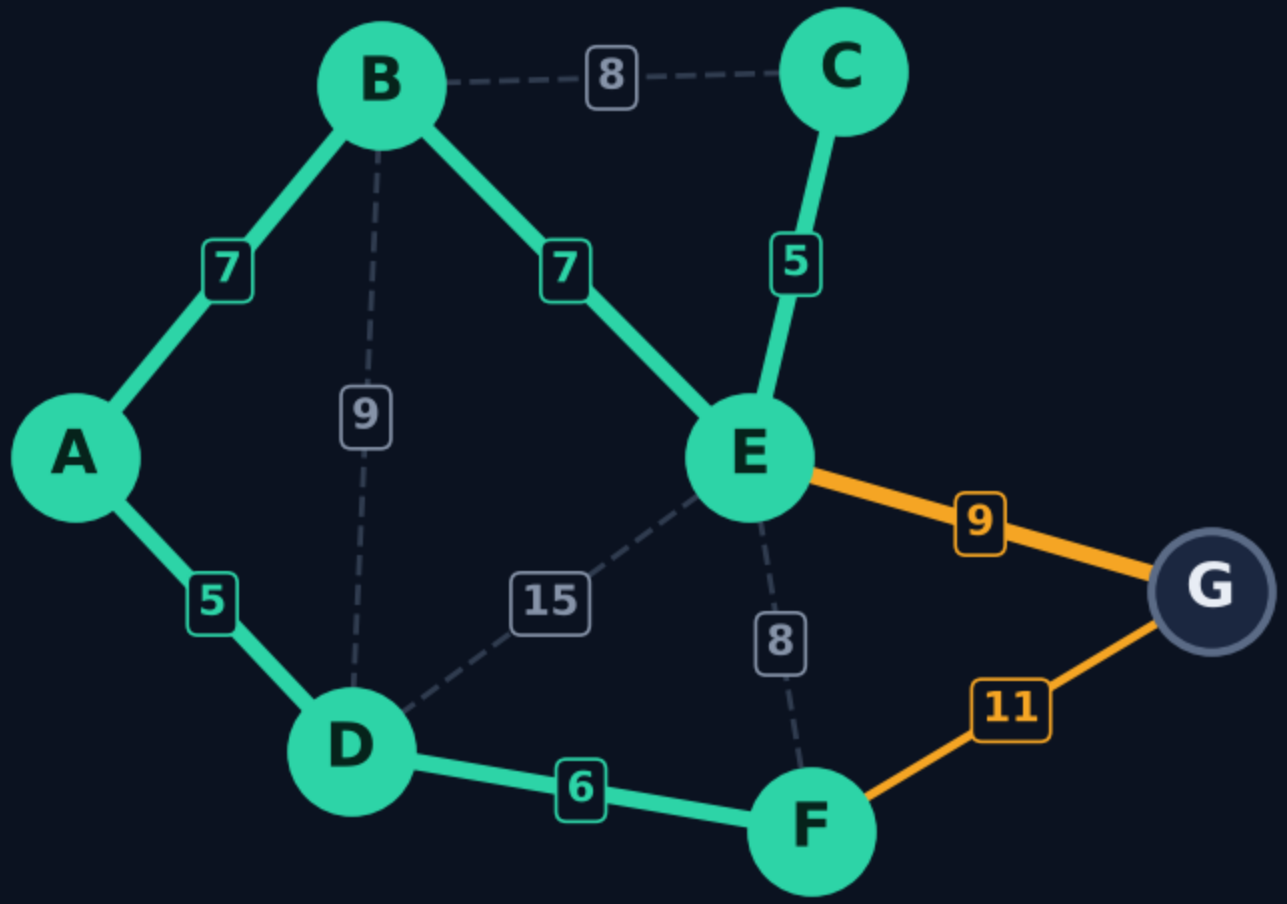
Skipped (would form a cycle):

B–C (8)

Total weight so far:

30

We add C to E. C joins the tree, and B to C is skipped. Total: thirty.



Step 6: look

Tree: A, D, F, B, E, C

Frontier edges:

E–G 9 ← cheapest

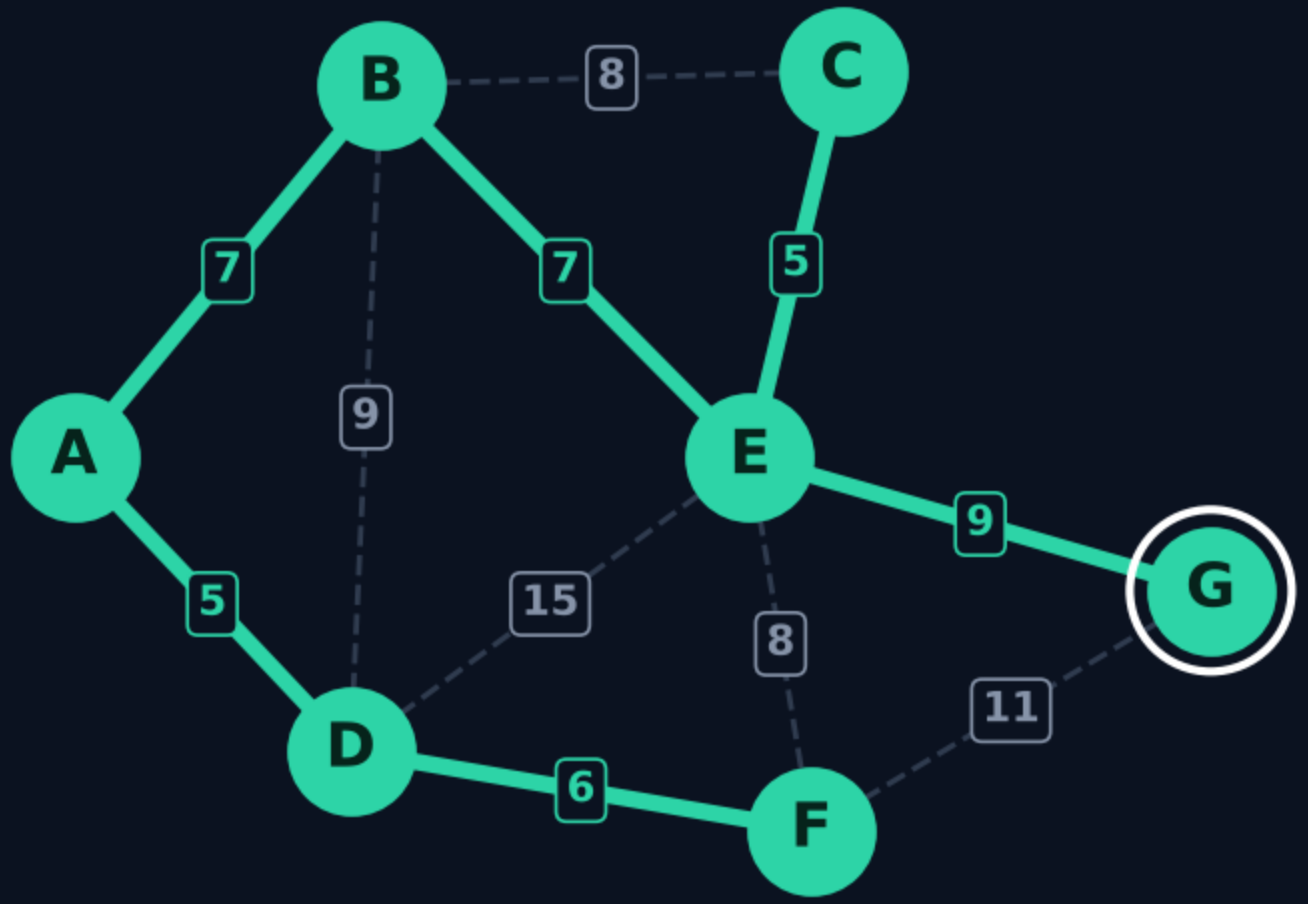
F–G 11

Step six. Only two edges are left: E to G, weight nine, and F to G, weight eleven. The cheapest is E to G.

Prim's Algorithm

Tree 7/7

Step 6 of 6



Step 6: add

Add edge E–G (weight 9)

Add vertex G to the tree

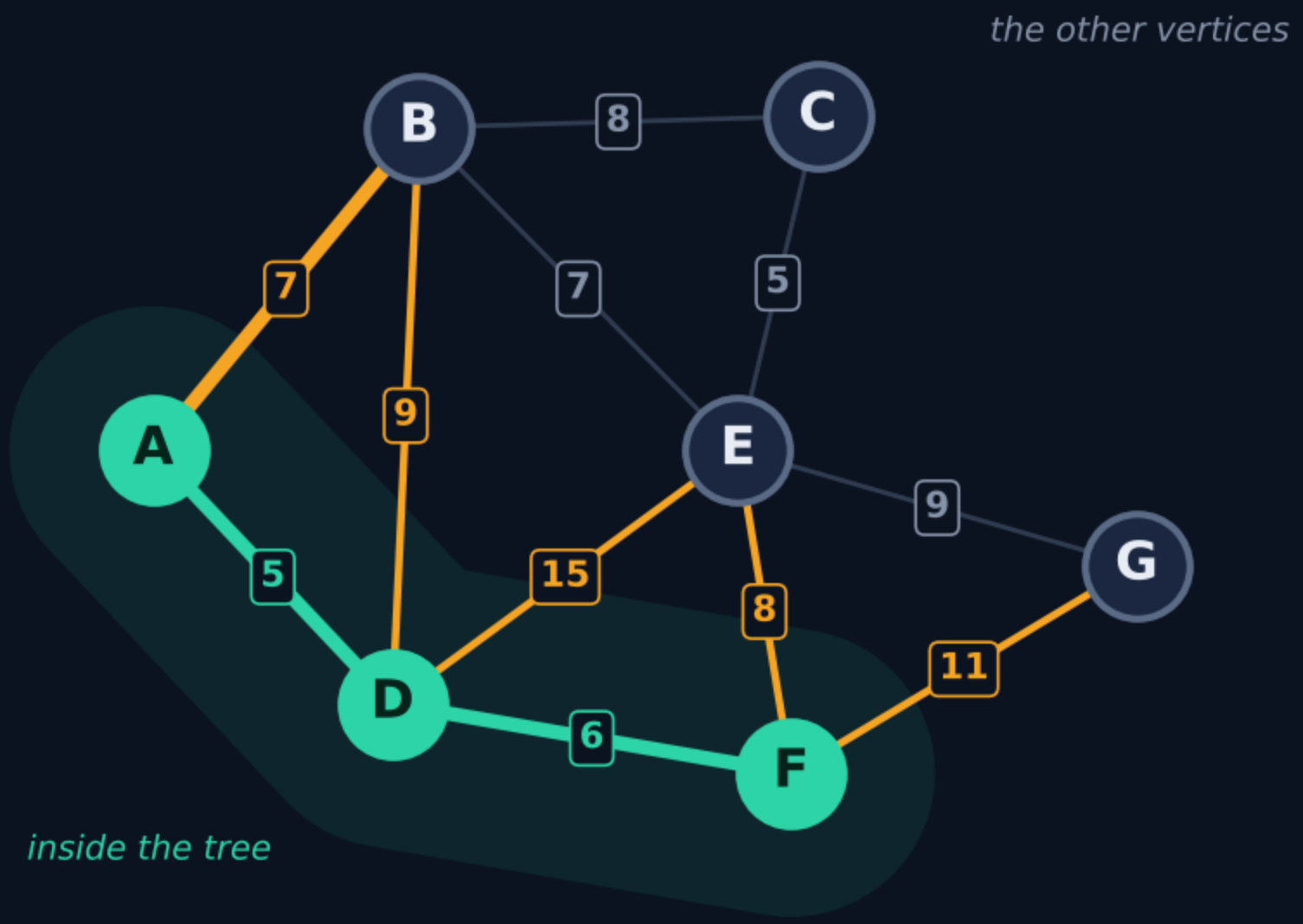
Skipped (would form a cycle):

F–G (11)

Total weight so far:

39

We add E to G. Every vertex is in the tree, so we stop. The total weight is thirty nine.



The cut property

Split the vertices into two groups: tree | rest.

Every frontier edge crosses that split.

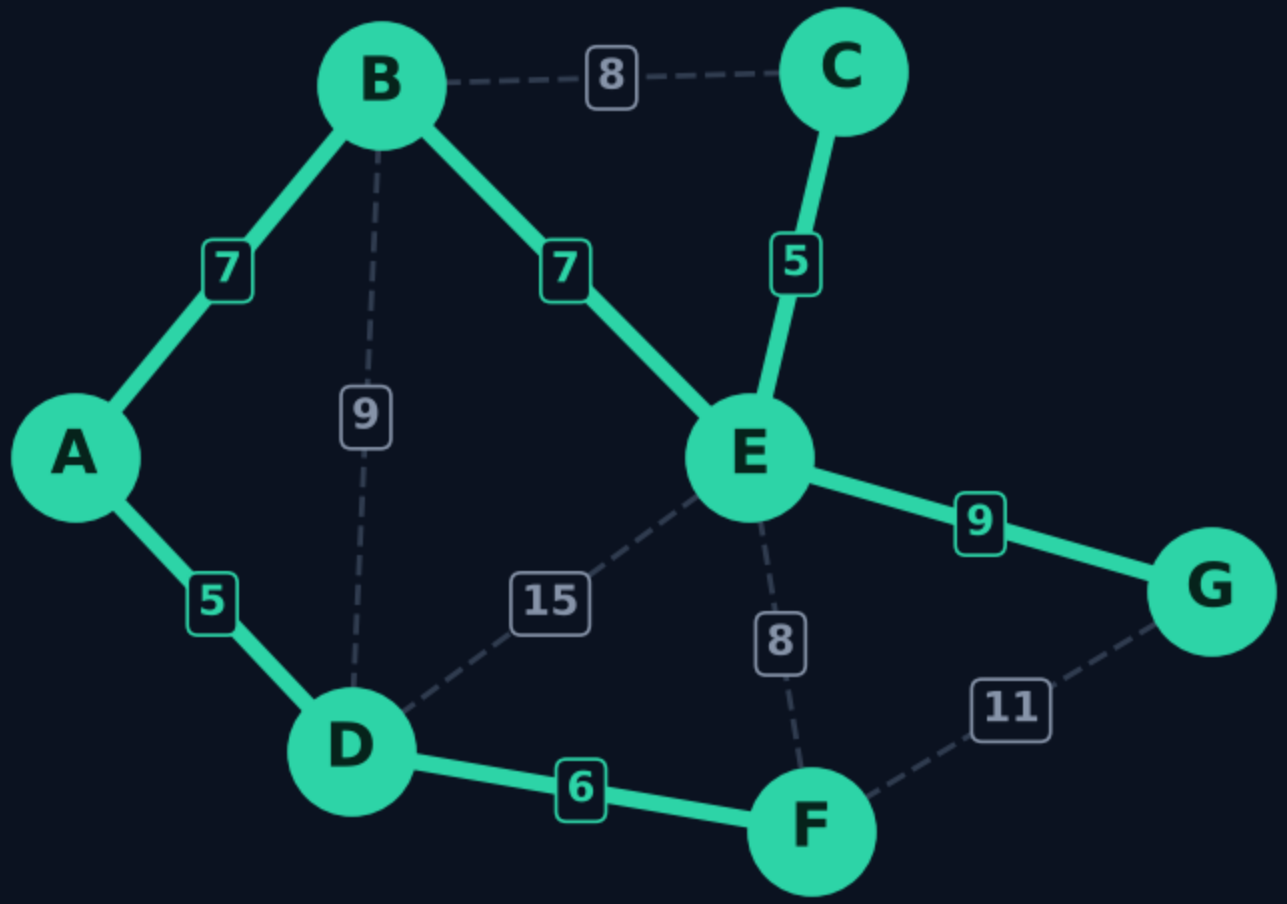
The cheapest crossing edge is always safe.

Why does being greedy work? At every step, the tree and the other vertices form a split. The cheapest edge across that split is always safe to keep. That is the cut property, and it is Prim's guarantee.

Prim's Algorithm

Tree 7/7

Done



Result: the MST

6 edges, 7 vertices

Total weight:

39

No spanning tree

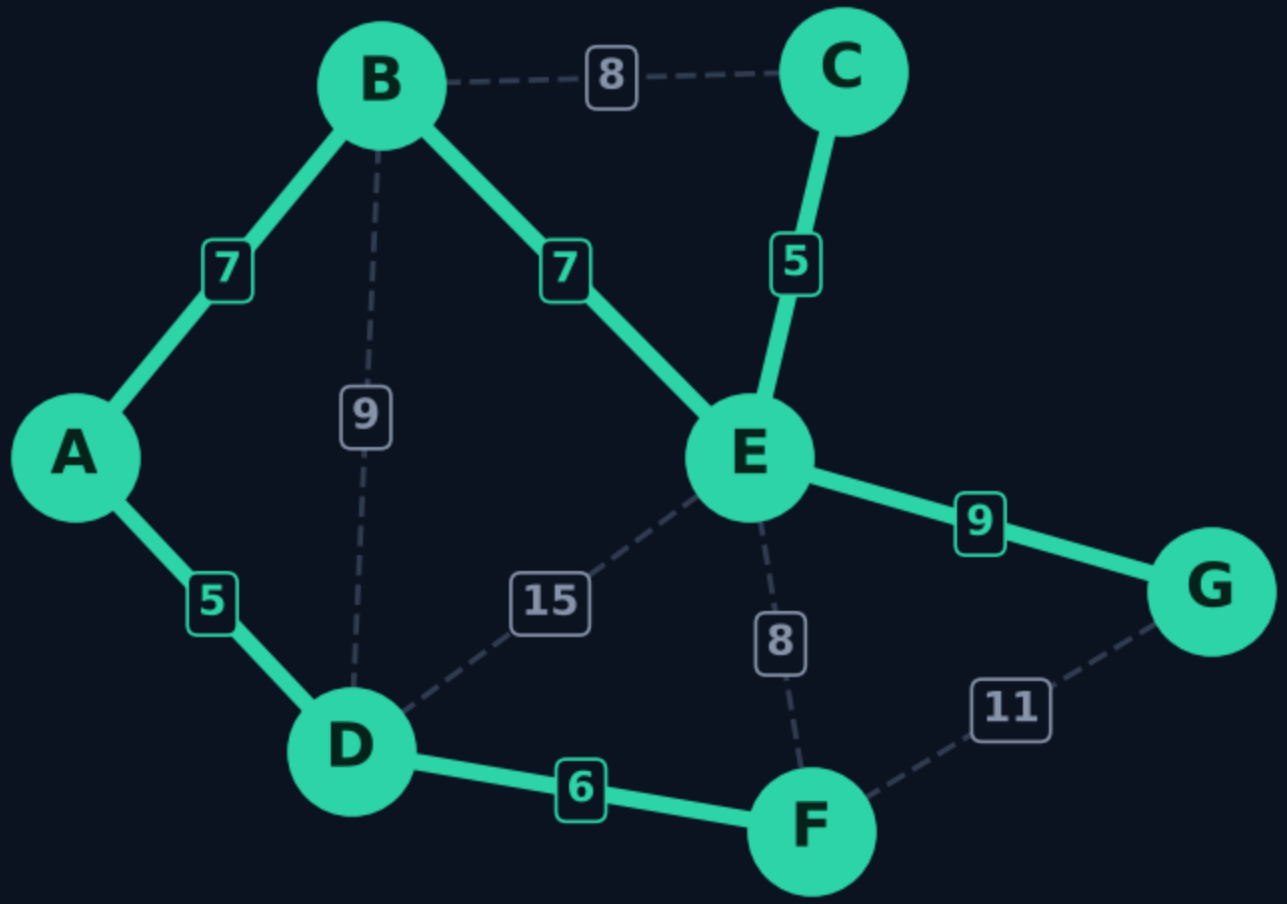
can be cheaper.

Here is the result. Six edges connect all seven vertices, with a total weight of thirty nine. No other spanning tree can do better.

Prim's Algorithm

Tree 7/7

Done



Time complexity

Binary heap

$O(E \log V)$

Adjacency matrix

$O(V^2)$

E = edges, V = vertices

With a binary heap, Prim's algorithm runs in big O of E log V time.
With a plain adjacency matrix, it runs in big O of V squared.

Where is it used?



Network design

Cables, fibre and power grids: link every site at the lowest cost.



Roads & pipelines

Plan the cheapest way to reach every town or facility.

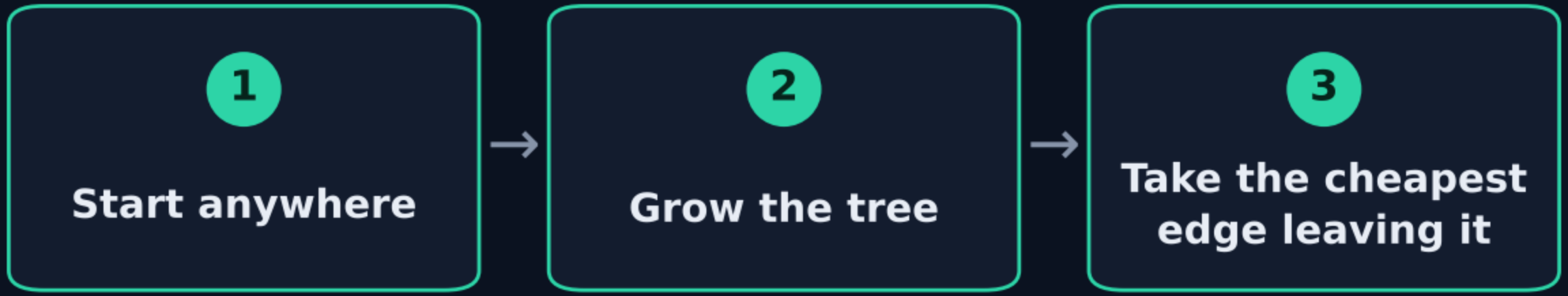


Data clustering

Cut the longest MST edges to split data into natural groups.

You will find this idea in network and cable design, in road and pipeline planning, and even in clustering data.

Remember



Thanks for watching!

To remember it: start anywhere, grow the tree, and always take the cheapest edge leaving it. Thanks for watching!

Narration script

1. Hi everyone! Today we'll see how Prim's algorithm finds a minimum spanning tree, one step at a time.
2. Imagine you have to connect these seven points with cables. Every possible cable has a cost, shown as a number on its edge.
3. You want every point connected, for the lowest total cost, with no wasteful loops. That cheapest network is called a minimum spanning tree.
4. Prim's algorithm is greedy. It grows a single tree, one edge at a time, using four simple moves.
5. One. Start from any vertex. That vertex is now our tree.
6. Two. Find the frontier: every edge that connects the tree to a vertex outside it.
7. Three. Pick the cheapest frontier edge, and add it to the tree, together with its new vertex.
8. Four. Repeat until every vertex is in the tree.
9. Let's try it. We start at vertex A. The tree is just A, and its total weight is zero.
10. Step one. The frontier has two edges: A to B, seven, and A to D, five. The cheapest is A to D.
11. We add A to D, and D joins the tree. Total weight so far: five.
12. Step two. Now there are four edges on the frontier. The cheapest is D to F, with weight six.
13. We add D to F, and F joins the tree. The total is now eleven.
14. Step three. Five edges leave the tree, and the cheapest is A to B, with weight seven.
15. We add A to B. Now B to D connects two vertices that are already in the tree, so it would create a cycle. We skip it. Total: eighteen.
16. Step four. Again five frontier edges, and the cheapest is B to E, with weight seven.
17. We add B to E. D to E and E to F are now inside the tree as well, so both are skipped. Total: twenty five.
18. Step five. Four frontier edges. Edge C to E, with weight five, is the cheapest.
19. We add C to E. C joins the tree, and B to C is skipped. Total: thirty.
20. Step six. Only two edges are left: E to G, weight nine, and F to G, weight eleven. The cheapest is E to G.
21. We add E to G. Every vertex is in the tree, so we stop. The total weight is thirty nine.

Narration script (continued)

22. Why does being greedy work? At every step, the tree and the other vertices form a split. The cheapest edge across that split is always safe to keep. That is the cut property, and it is Prim's guarantee.

23. Here is the result. Six edges connect all seven vertices, with a total weight of thirty nine. No other spanning tree can do better.

24. With a binary heap, Prim's algorithm runs in big O of E log V time. With a plain adjacency matrix, it runs in big O of V squared.

25. You will find this idea in network and cable design, in road and pipeline planning, and even in clustering data.

26. To remember it: start anywhere, grow the tree, and always take the cheapest edge leaving it. Thanks for watching!